Decision Trees

University of South Florida / Project 3

GGGamers

Alexander Bugar · Hector Colon
Liam Dzanan · Seth Wisniewski

What the program does

Our program loads a decision tree from a text file. You can follow its questions to a decision or browse individual nodes. The terminal draws the tree and highlights the path to the selected node. We included examples for buying a car and choosing an investment. Each successful load also writes a report to `about_tree.txt`.

Build and run on the student cluster

From the `Project 3` folder, use CMake and a C++17 compiler:

```
cmake -S . -B build
cmake --build build
./build/decision_tree
```

The interface uses FTXUI, stored in `third_party/ftxui`.

Open a tree

Enter `tree-car.txt`, `tree-investment.txt`, or a full file path and press Enter. If the file does not load, check the filename and try again.

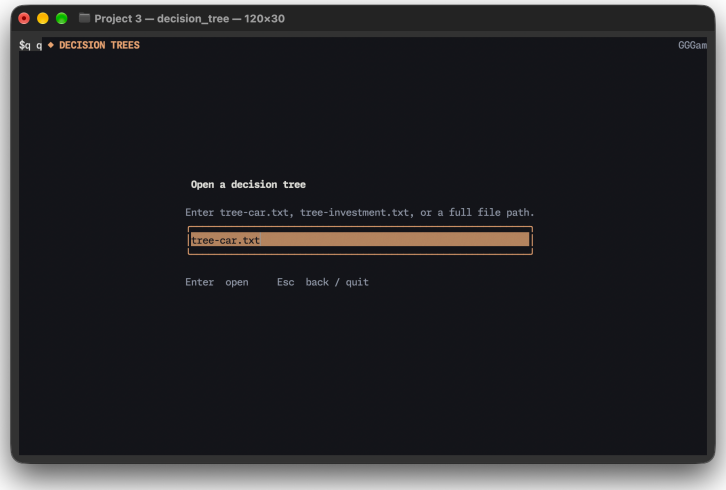


Figure 1. The filename field before loading the car example.

Using the interface

After a file loads, choose **Walk through questions** to answer one question at a time, or **Explore all nodes** to browse the tree in preorder. Answer choices come from the file, including Sometimes in the car example.

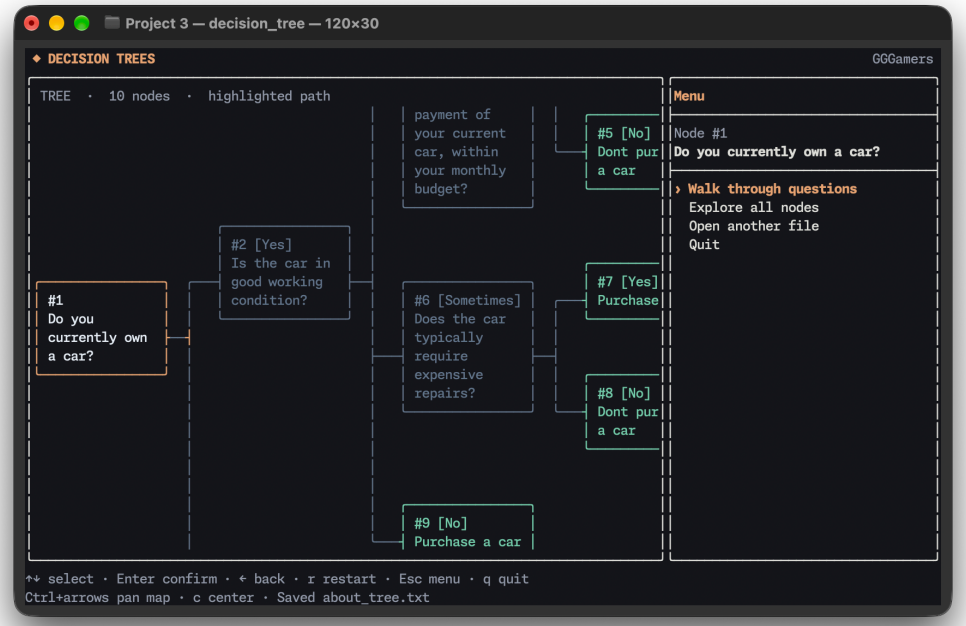


Figure 2. The main menu with the car tree loaded.

Some branches are offscreen in Figure 2. Use Ctrl + arrow keys to scroll or make the terminal larger. The map centers on the selected node and is hidden when the terminal is too small.

Keyboard reference

Key	Action
Up / down	Select an answer, menu item, or node.
Enter	Confirm a choice; in Explore, begin walking from that node.
Left / Backspace	Move to the parent while walking or exploring.
Right	Move to the first child in Explore.
`r` / Home	Return to the root while walking or exploring.
Ctrl + arrows	Scroll the diagram.
`c`	Center the diagram on the selected node.
Esc	Return to the menu; quit from the menu or before a tree loads.
`q`	Quit, except when typing in the filename field.

How the tree is stored

Each node stores its position, answer label, text, parent pointer, first-child pointer, and next-sibling pointer. Siblings form a linked list, which lets a question have more than two answers. For example, node 2 in the car tree has three children.

The program sorts the records by preorder position before linking the nodes. It keeps a list of ancestors while connecting each node to the parent one level above it. Invalid positions or levels are rejected. When the tree is destroyed, it deletes each child's subtree before deleting the parent.

How the diagram is drawn

The interface gets a preorder copy of the nodes, with indexes for parents and children. In `ui/TreeDiagram.cpp`, the drawing steps are:

1. Find each node's depth and wrap its text to fit a box.
2. Work backward through the list to reserve space for each subtree.
3. Place parents beside their children, with one column per depth.
4. Center the view on the selection and mark its path back to the root.
5. Draw connections, borders, and text inside the visible area.

`LinkedTree.cpp` handles tree operations. `ui/TreeView.cpp` handles the keyboard and menus.

Report and sample results

Loading a tree replaces `about_tree.txt` in the folder where the program was started. It contains an indented tree, the root, node counts, height, preorder node lists, and binary-tree checks. Height counts edges, so a leaf has height zero.

Property	Car tree	Investment tree
Internal nodes	4	3
Leaf nodes	6	4
Height	3	3
Binary	No	Yes
Proper	Not applicable	Yes
Perfect	Not applicable	No
Balanced	Not applicable	No

A binary tree has at most two children per node. A proper binary tree has exactly two children per internal node. A perfect tree is proper with all leaves at the same depth. A binary tree is balanced when the left and right subtree heights differ by at most one at every node. The program reports these last three checks only for binary trees.

Plain console option

Run `./build/decision_tree --plain` and enter a node position to see its text, ancestor, descendant, and sibling. Missing relatives show `None`. Type `exit` to quit. To build the console directly from `Project 3`:

```
g++ -std=c++17 main.cpp LinkedTree.cpp -o decision_tree
./decision_tree
```

Example: reaching a decision

In the car example, choosing Yes at nodes 1, 2, and 3 leads to node 4: **Purchase a car**. Orange marks the selected path, and green marks leaves outside that path. The node numbers match the preorder positions in the input file.

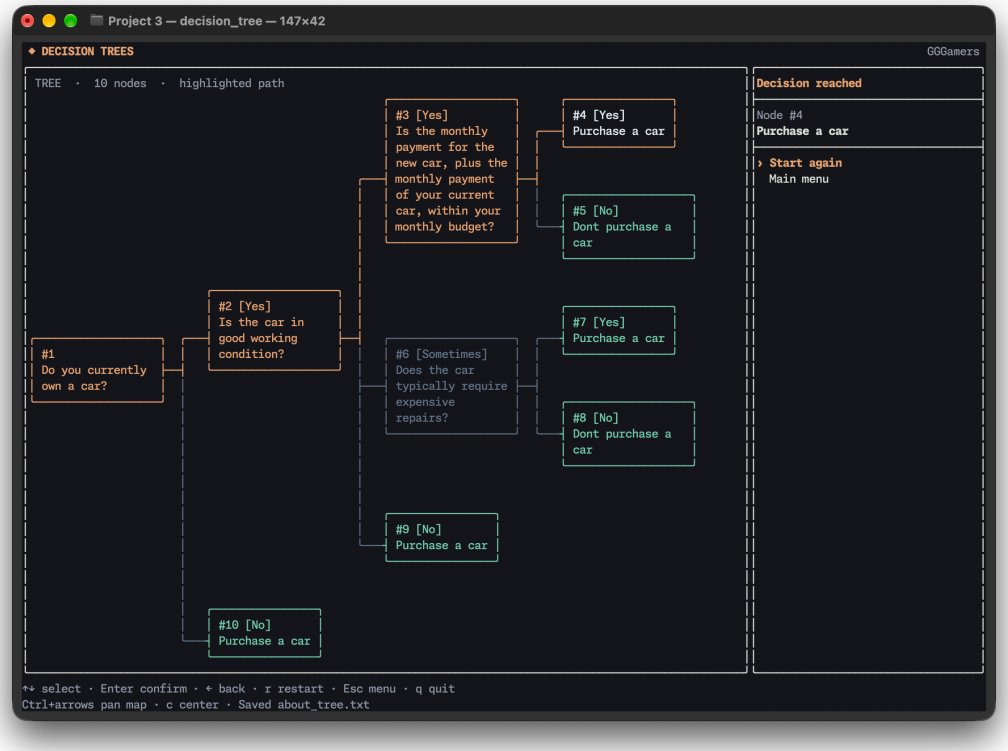


Figure 3. Choosing Yes three times leads to Purchase a car.

Choose **Start again** to return to the first question or **Main menu** to return to the menu. The investment example uses the same controls, but every question has two answers. It is proper because every internal node has two children. It is not perfect because its leaves have different depths, and it is not balanced because the root's subtree heights differ by two.

Distribution of work

Alexander Bugar	25%
Hector Colon	25%
Liam Dzanan	25%
Seth Wisniewski	25%

Academic integrity statement

We understand that there will be no tolerance towards academic dishonesty, and that cheating will lead to an academic referral. We are aware of the identified behaviors that are considered violations of the academic standards for Undergraduate and Graduate students per USF policy.